στο αποτέλεσμα της αναζήτησης, ο κώδικας που την καλεί θα βασίσει την περαιτέρω ροή του. Για παράδειγμα, ενδέχεται αν η τιμή βρεθεί, ο χρήστης της συνάρτησης να επιθυμεί να αυξήσει την τιμή στον πίνακα κατά 1. Αν πάλι η τιμή δεν βρεθεί, μπορεί να μην χρειάζεται καμία περαιτέρω ενέργεια για τον συγκεκριμένο πίνακα.

Το πρόβλημα αυτό λύνεται με την τιμή επιστροφής της συνάρτησης. Στο συγκεκριμένο παράδειγμα μια καλή τιμή επιστροφής είναι τύπου int. Στην περίπτωση που θα βρεθεί η τιμή αναζήτησης, ο ακέραιος που θα επιστραφεί θα αναπαριστά τη θέση της τιμής αναζήτησης στον πίνακα. Αν πάλι η τιμή δεν βρεθεί, μπορεί να επιστραφεί η τιμή -1. Καθώς δεν υπάρχει θέση του πίνακα με αρνητική τιμή, η τιμή -1 μπορεί να αναπαραστήσει με ασφάλεια την αποτυχία της αναζήτησης. Προσέξτε εδώ πως αν διαλέξουμε να αναπαραστήσουμε την αποτυχία αναζήτησης με μη αρνητικό αριθμό, π.χ. με 0, τότε έχουμε κακό σχεδιασμό της συνάρτησης. Αυτό γιατί η συνάρτηση θα επιστρέφει 0, τόσο στην περίπτωση που το στοιχείο αναζήτησης δεν βρεθεί, όσο και στην περίπτωση που βρεθεί στη θέση 0 του πίνακα.

Επομένως, η διεπαφή της συνάρτησης τροποποιείται όπως δείχνει ο ακόλουθος κώδικας:

```
public static int sequentialSearch(int[] array, int
schElement)
```

Η τιμή επιστροφής έγινε int από void που ήταν στην προηγούμενη έκδοση. Τώρα η συνάρτηση μπορεί να κληθεί όπως δείχνει ο κώδικας 7.2:

```
static void callSeqSearch() { //Κώδικας 7.2
    int[] t1 = {3, 2, 5, 3, 6, 9};
    int sItem = 5;
    int idx = sequentialSearch(t1, sItem);
    if (idx > -1) {
        t1[idx]++;
    }
}
```

Κώδικας 7.2 *Κλήση της sequentialSearch με παραμέτρους και τιμή επιστροφής*

Στον κώδικα 7.2, η κλήση της συνάρτησης επιστρέφει τη θέση του πίνακα στην οποία βρέθηκε το sItem ή -1. Αν επιστρέψει -1, καμία ενέργεια επάνω στον πίνακα δεν γίνεται. Αν όμως επιστρέψει μη αρνητική τιμή, τότε προσαυξάνεται η τιμή στη θέση που επιστράφηκε κατά 1.

Ακόμη και αν κάνουμε τις αλλαγές που συζητήσαμε μέχρι τώρα, παραμένει ένα σοβαρό μειονέκτημα στη συνάρτηση. Αυτό συνίσταται στο γεγονός ότι η συνάρτηση εμφανίζει τα αποτελέσματα της αναζήτησης. Προσέξτε, το μειονέκτημα δεν είναι ότι επιστρέφει στον κώδικα που την καλεί τα αποτελέσματα της αναζήτησης αλλά ότι τα εμφανίζει χρησιμοποιώντας μάλιστα την println. Αυτό το μειονέκτημα πολλές φορές γίνεται δύσκολα κατανοητό στους αρχάριους προγραμματιστές. Πράγματι, αν κάνω έναν κώδικα που ψάχνει σε συγκεκριμένο πίνακα για να βρει μια συγκεκριμένη τιμή και το μόνο που θέλω είναι να μάθω αν η τιμή βρέθηκε και που, τότε δεν είναι πρόβλημα αν ο κώδικας εμφανίζει το αποτέλεσμα της αναζήτησης. Μια συνάρτηση όμως μπορεί να κληθεί σε ένα σημείο που θέλω να εμφανίσω το αποτέλεσμά της αλλά μπορεί να κληθεί και σε πολλά σημεία που δεν θέλω να το εμφανίζω αλλά απλώς το χρειάζομαι μέσα στον κώδικά μου. Για παράδειγμα, μπορεί την sequentialSearch να την καλέσω σε μια παραθυρική εφαρμογή. Σε αυτήν την περίπτωση η έξοδος με την println που είναι κατάλληλη μόνο για περιβάλλοντα γραμμής εντολών (command line) προφανώς θα μου καταστρέψει τη διεπαφή χρήστη (user interface).

Ο κώδικας 7.3 δίνει τον ορισμό της sequentialSearch που ενσωματώνει όλες τις παρατηρήσεις μέχρι εδώ.

```
public static int sequentialSearch(int[] array, int sItem) {
    for (int i = 0; i < array.length; i++) {
        if (array[i] == sItem) {
```

```
                return i;
            }
        }
        return -1;
    }
```

Κώδικας 7.3 *Συνάρτηση σειριακής αναζήτησης σε μονοδιάστατο πίνακα ακέραιων*

Η sequentialSearch σε αυτή τη μορφή μπορεί να κληθεί από οποιονδήποτε κώδικα για να αναζητήσει σε οποιονδήποτε μονοδιάστατο πίνακα ακέραιων οποιαδήποτε ακέραια τιμή. Προσέξτε πως η επιστροφή τιμής από τη συνάρτηση γίνεται με χρήση της δεσμευμένης λέξης return. Η λέξη return συντάσσεται με έκφραση που ο τύπος της είναι υποχρεωτικά ίδιος ή συμβατός με τον τύπο της συνάρτησης. Η return προκαλεί άμεση έξοδο από τη συνάρτηση. Σε συναρτήσεις τύπου void, η return συντάσσεται μόνη, ακολουθούμενη απλώς από τον χαρακτήρα ελληνικό ερωτηματικό (semicolon). Σε void συναρτήσεις, η return δεν είναι υποχρεωτική. Ωστόσο, σε κάποιες περιπτώσεις είναι χρήσιμη.

Οι παράμετροι των συναρτήσεων στην Java είναι παράμετροι θέσης (positional parameters). Αυτό σημαίνει πως κατά την κλήση συνάρτησης, οι πραγματικές παράμετροι αντιστοιχίζονται με τις τυπικές παραμέτρους ανάλογα με τη θέση που διατηρεί κάθε μία στην λίστα παραμέτρων.

Τέλος να επισημάνουμε πως οι συναρτήσεις στα πλαίσια του αντικειμενοστρεφούς προγραμματισμού ονομάζονται και μέθοδοι (methods).

7.2 Υπερφόρτωση Συναρτήσεων

Πάντως, είναι πιθανό να χρειάζεται να κάνουμε αναζήτηση τιμών και σε πίνακες που δεν είναι πίνακες ακέραιων. Ας υποθέσουμε πως χρειάζεται να κάνουμε αναζήτηση σε έναν πίνακα πραγματικών. Σε αυτήν την περίπτωση μπορούμε να υλοποιήσουμε μια άλλη συνάρτηση με παραμέτρους τύπου double[] και double. Το θέμα είναι πως παρότι αλλάζουν οι τύποι των δεδομένων, η λειτουργία παραμένει ίδια. Δεν θέλουμε για την ίδια λειτουργία να έχουμε συναρτήσεις με διαφορετικό όνομα. Παρόλα αυτά, για κάποιες γλώσσες προγραμματισμού, η μόνη λύση είναι να κάνουμε μια συνάρτηση που έχει διαφορετικό όνομα από τη συνάρτηση που ψάχνει σε πίνακες ακέραιων. Αυτό όμως δεν είναι βολικό γιατί δεν θέλουμε να αναγκαζόμαστε να εφευρίσκουμε και να θυμόμαστε διαφορετικά ονόματα για την ίδια λειτουργία. Ευτυχώς, η Java όπως και οι περισσότερες σύγχρονες γλώσσες προγραμματισμού, επιλύουν αυτό το θέμα με την υπερφόρτωση συναρτήσεων (function overloading).

Με την υπερφόρτωση συναρτήσεων μπορούμε να ορίσουμε συναρτήσεις με το ίδιο όνομα αρκεί να διαφέρει η ταυτότητά τους (function signature). Η ταυτότητα μιας συνάρτησης αποτελείται από το όνομα της συνάρτησης και τη λίστα παραμέτρων. Προσέξτε πως η τιμή επιστροφής δεν συμπεριλαμβάνεται στην ταυτότητα. Επομένως, μπορούμε να κάνουμε συναρτήσεις με το ίδιο όνομα, αρκεί να διαφέρει η λίστα παραμέτρων. Ποια από δύο συναρτήσεις που έχουν το ίδιο όνομα θα κληθεί είναι μια απόφαση που λαμβάνει ο μεταγλωττιστής ανάλογα με τις πραγματικές παραμέτρους. Στον κώδικα 7.4 δίνουμε παράδειγμα ορισμού και χρήσης υπερφορτωμένων συναρτήσεων.

```
static void f(int i) { //Κώδικας 7.4
      System.out.println("f(int)");
}

static void f(double d) { //Κώδικας 7.4
      System.out.println("f(double)");
}

public static void main(String[] args) { //Κώδικας 7.4
      f(1);
      f(1d);
}
```

Κώδικας 7.4 *Ορισμός και χρήση υπερφορτωμένων συναρτήσεων*

Στον κώδικα 7.4 και οι δύο συναρτήσεις που ορίζουμε ονομάζονται f. Η μία όμως δέχεται μια ακέραια παράμετρο και η άλλη μια πραγματική παράμετρο. Στην main καλούμε την f την πρώτη φορά με ακέραια παράμετρο και τη δεύτερη με πραγματική. Η έξοδος του κώδικα είναι:

```
f(int)
f(double)
```

Η έξοδος αυτή μας δείχνει πως την πρώτη φορά κλήθηκε η f που λαμβάνει την ακέραια παράμετρο και τη δεύτερη φορά κλήθηκε η f που λαμβάνει την πραγματική παράμετρο.
Έτσι, αν θέλουμε να έχουμε και μια sequentialSearch ώστε να ψάχνει και σε πραγματικούς πίνακες δεν έχουμε παρά να την υπερφορτώσουμε όπως δείχνει ο κώδικας 7.5.

```
public static int sequentialSearch(double[] array, double sItem) {
    for (int i = 0; i < array.length; i++) {
        if (approximateEquals(array[i], sItem, 0.0001)) {
            return i;
        }
    }
    return -1;
}
```

Κώδικας 7.5 *Σειριακή αναζήτηση σε πίνακες πραγματικών*

7.3 Παράμετροι

Έχουμε ήδη συζητήσει σχετικά με τις μεταβλητές αναφοράς και τις μεταβλητές τιμής. Η συμπεριφορά αυτών των δύο τύπων μεταβλητών είναι διαφορετική όταν περνάνε σαν παράμετροι σε συναρτήσεις. Σε αυτήν την ενότητα εξετάζουμε πρώτα τις μεταβλητές τιμής ως παραμέτρους και στη συνέχεια τις μεταβλητές αναφοράς.

7.3.1 Οι μεταβλητές τιμής ως παράμετροι

Ο κώδικας 7.6 παρουσιάζει δύο συναρτήσεις, την increment και την main. Μελετήστε τον κώδικα και επιχειρήστε να υποθέσετε την έξοδό του.

```
static void increment(int i) {
    i++;
    System.out.println(i);
}

public static void main(String[] args) {
    int i = 0;
    increment(i);
    System.out.println(i);
}
```

Κώδικας 7.6 *Παράδειγμα παραμέτρων τιμής*

Η συνάρτηση increment λαμβάνει ως παράμετρο έναν ακέραιο. Το αναγνωριστικό της παραμέτρου είναι i. Στην πρώτη γραμμή αυξάνει την τιμή της παραμέτρου κατά 1 και στη συνέχεια την τυπώνει.

Η main ορίζει μια ακέραια μεταβλητή, επίσης με το αναγνωριστικό i και την αρχικοποιεί στο 0. Στη συνέχεια καλεί την increment με πραγματική παράμετρο το i. Μετά το πέρας της increment, η main τυπώνει την i.

Πρώτη σημαντική παρατήρηση είναι ότι η i της increment και η i της main δεν ταυτίζονται. Μπορεί να είναι και οι δύο τύπου int και να έχουν το ίδιο αναγνωριστικό, ωστόσο πρόκειται για διαφορετικές μεταβλητές. Εξάλλου θα μπορούσαν να έχουν διαφορετικά αναγνωριστικά. Αν μεταβάλλετε το αναγνωριστικό στην main, π.χ. το κάνετε j, θα διαπιστώσετε πως δεν θα έχετε καμία διαφορά στο αποτέλεσμα του κώδικα.

Αν τρέξουμε τον κώδικα, θα διαπιστώσουμε πως η μεν increment τυπώνει 1, η δε main τυπώνει 0. Αυτή η συμπεριφορά εξηγείται ως εξής: Η main δεσμεύει χώρο στη μνήμη κατάλληλο για τη μεταβλητή i, δηλαδή χώρο κατάλληλο για έναν ακέραιο. Σε αυτήν τη θέση μνήμης εκχωρείται η τιμή 0. Όταν καλείται η increment, ένα αντίγραφο της πραγματικής παραμέτρου περνάει στην τυπική παράμετρο. Στη συνέχεια η i της increment λειτουργεί σαν τοπική μεταβλητή. Η increment δηλαδή δεσμεύει επίσης χώρο για έναν ακέραιο στον οποίο αντιγράφεται η τιμή της πραγματικής παραμέτρου. Από εκεί και μετά, η increment επενεργεί επάνω στη δική της μεταβλητή i που είναι εντελώς ανεξάρτητη από την i της main. Με το τέλος εκτέλεσης της increment, ο έλεγχος επανέρχεται στην main. Η μεταβλητή i της main έχει διατηρήσει την τιμή της και έτσι εμφανίζεται το 0.

Οι μεταβλητές όλων των θεμελιωδών τύπων στην Java είναι μεταβλητές τιμής και περνούν στις συναρτήσεις σαν παράμετροι τιμής.

7.3.2 Οι μεταβλητές αναφοράς ως παράμετροι

Είπαμε παραπάνω πως οι θεμελιώδεις τύποι περνάνε στις συναρτήσεις σαν παράμετροι τιμής. Το ίδιο ισχύει και για τις μεταβλητές αναφοράς. Ωστόσο η συμπεριφορά είναι διαφορετική. Μελετήστε τον κώδικα 7.7. Ποια είναι κατά την γνώμη σας η έξοδός του;

```
static void increment(int[] store) { //Κώδικας 7.7
    for (int i = 0; i < store.length; i++) {
        store[i]++;
    }
    System.out.println(Arrays.toString(store));
}

public static void main(String[] args) {
    int[] s = {1, 2, 3};
    increment(s);
    System.out.println(Arrays.toString(s));
}
```

Κώδικας 7.7 *Η μεταβλητή αναφοράς ως παράμετρος*

Η έξοδος του κώδικα 7.7 είναι

```
[2, 3, 4]
[2, 3, 4]
```

Αυτό σημαίνει πως όποιες αλλαγές έγιναν στον πίνακα μέσα στην increment ισχύουν και έξω από αυτήν. Η εξήγηση έχει ως εξής: Η μεταβλητή s είναι μία αναφορά που περιέχει τη διεύθυνση του πίνακα. Η αναφορά αυτή αντιγράφεται στην παράμετρο store της increment. Εφόσον η s είναι αντίγραφο της store, και οι δύο δείχνουν στην ίδια θέση μνήμης. Άρα η increment επενεργεί στον ίδιο πίνακα στον οποίο αναφέρεται η s. Όποια μεταβολή γίνει στα στοιχεία του πίνακα μέσω μιας αναφοράς σε αυτόν, ισχύει όταν τον προσπελάσουμε μέσω οποιασδήποτε άλλης αναφοράς.

Προσέξτε όμως μια άλλη επίπτωση. Μελετήστε τον κώδικα 7.8. Ποια είναι η έξοδός του κατά την άποψή σας;

```
static void assign(int[] store) { //Κώδικας 7.8
    int[] s = {4, 5, 6};
```

```
        store = s;
        System.out.println(Arrays.toString(store));
    }

    public static void main(String[] args) { //Κώδικας 7.8
        int[] table = {1, 2, 3};
        assign(table);
        System.out.println(Arrays.toString(table));
    }
```

Κώδικας 7.8 *Η μεταβλητή αναφοράς ως παράμετρος τιμής*

Η έξοδος του κώδικα 7.8 έχει ως εξής:

```
[4, 5, 6]
[1, 2, 3]
```

Η έξοδος αυτή μας δείχνει πως μέσα στην assign, η store μεταβλήθηκε ώστε να δείχνει στον ίδιο πίνακα με την τοπική μεταβλητή s. Ωστόσο, η table συνεχίζει να αναφέρεται στον πίνακα που δημιουργήθηκε στην main. Στην πράξη, ένα αντίγραφο της table πέρασε στην store. Το αντίγραφο, δηλαδή η store, ενημερώθηκε ώστε να δείχνει σε άλλο πίνακα, όμως η table έμεινε ανέπαφη. Το ίδιο ακριβώς αποτέλεσμα θα είχαμε βέβαια αν ονομάζαμε store την table.

Συνοψίζοντας, σε σχέση με τις μεταβλητές αναφοράς ως παραμέτρους, ισχύει το εξής στην Java: Όταν μια μεταβλητή αναφοράς περνάει σαν παράμετρος σε συνάρτηση, η συνάρτηση μπορεί να μεταβάλει το περιεχόμενο της μνήμης στην οποία αναφέρεται η εν λόγω μεταβλητή, δεν μπορεί όμως να μεταβάλει το περιεχόμενο της ίδιας της μεταβλητής.

7.3.3 Λίστα παραμέτρων μεταβλητού μήκους

Ας υποθέσουμε τώρα πώς αναπτύσσουμε μια εφαρμογή στην οποία απαιτείται συχνά να υπολογίζουμε τον μέσο όρο μιας σειράς πραγματικών τιμών. Το θέμα όμως είναι πως σε ένα σημείο του κώδικα χρειάζεται να υπολογίσουμε τον μέσο όρο τεσσάρων πραγματικών τιμών, σε κάποιο άλλο σημείο χρειάζεται ο μέσος όρος επτά πραγματικών τιμών. Μία λύση θα ήταν να υλοποιήσουμε δύο συναρτήσεις. Η μία με 4 τυπικές παραμέτρους και η άλλη με επτά. Αν αργότερα χρειαστεί να υπολογίσουμε τον μέσο όρο πέντε πραγματικών τιμών, τότε θα πρέπει να προσθέσουμε τον ορισμό κατάλληλης συνάρτησης με πέντε τυπικές παραμέτρους. Η αντιμετώπιση αυτή όμως είναι προβληματική. Σε όλες τις περιπτώσεις, ο αλγόριθμος είναι ο ίδιος και το μόνο που αλλάζει είναι το πλήθος των δεδομένων.

Ευτυχώς για μας, η Java προσφέρει μια αποτελεσματική δυνατότητα, τη λίστα παραμέτρων μεταβλητού μήκους (variable length parameter list).

Ας δούμε στην πράξη πώς υλοποιείται και πώς αξιοποιείται μια συνάρτηση που υπολογίζει τον μέσο όρο μιας σειράς πραγματικών αριθμών.

```
static double average(double... sequence) {
    double sum = 0;
    for (int i = 0; i < sequence.length; i++) {
        sum += sequence[i];
    }
    return sum / sequence.length;
}

public static void main(String[] args) {
    double d1 = 12, d2 = 13.5, d3 = 16.1;
    double av1 = average(d1, 14.5);
    double av2 = average(d1, d2, d3);
    double totalAv = average(av1, av2);
```

```
        DecimalFormat f = new DecimalFormat("###.##");
        System.out.println(f.format(av1) + " " + f.format(av2) + " " +
f.format(totalAv));
    }
```

Κώδικας 7.9 *Παράμετρος μεταβλητού μήκους*

Η συνάρτηση average στον κώδικα 7.9 περιέχει την παράμετρο μεταβλητού μήκους sequence. Οι τρεις τελείες που ακολουθούν τον τύπο double δηλώνουν πως πρόκειται για παράμετρο μεταβλητού μήκους. Μόνο μια παράμετρος μεταβλητού μήκους επιτρέπεται σε μία συνάρτηση. Αν μάλιστα η συνάρτηση έχει και άλλες παραμέτρους, τότε η παράμετρος μεταβλητού μήκους πρέπει να βρίσκεται τελευταία στη λίστα παραμέτρων.

Στην main, η average καλείται τρεις φορές. Την πρώτη με δύο πραγματικές παραμέτρους, μία μεταβλητή και μία σταθερά, τη δεύτερη με τρεις πραγματικές παραμέτρους και την τρίτη φορά καλείται πάλι με δύο πραγματικές παραμέτρους.

Μελετώντας την υλοποίηση της average μπορούμε να διαπιστώσουμε πως οι παράμετροι περνάνε στην πράξη σε έναν πίνακα τύπου double. Εκείνο δηλαδή που συμβαίνει είναι πως ο μεταγλωττιστής τοποθετεί τις τιμές των πραγματικών παραμέτρων σε κατάλληλο πίνακα και περνάει μια αναφορά σε αυτόν τον πίνακα στην average. Έτσι μέσα στη συνάρτηση έχουμε πρόσβαση στις τιμές των πραγματικών παραμέτρων μέσω της average την οποία διαχειριζόμαστε κανονικά ως τυπική αναφορά σε μονοδιάστατο πίνακα πραγματικών. Εξαιτίας αυτού του μηχανισμού, μπορούμε να καλέσουμε την average με πραγματική παράμετρο έναν πίνακα πραγματικών όπως δείχνει ο κώδικας που ακολουθεί:

```
double[] d={12,13.5,16.1};
double totalAv=average(d);
```

7.3.4 Οι παράμετροι της main

Μια εφαρμογή Java μπορεί να κληθεί από διάφορα περιβάλλοντα. Στην τυπική περίπτωση, οι εφαρμογές Java σε αυτό το βιβλίο καλούνται μέσα από το NetBeans. Ωστόσο, μια εφαρμογή Java μπορεί να κληθεί μέσα από πολλά και διαφορετικά περιβάλλοντα. Για παράδειγμα, μπορεί να κληθεί από το powershell των Windows ή μέσα από μια άλλη εφαρμογή Java ή μέσα από οποιοδήποτε λειτουργικό στο οποίο τρέχει η JVM.

Επιπλέον, μια εφαρμογή θα πρέπει να έχει τρόπο ακριβώς όπως οι συναρτήσεις να λαμβάνει μια σειρά από παραμέτρους ώστε να διαφοροποιεί τη συμπεριφορά της ανάλογα με τις τιμές των παραμέτρων. Ωστόσο, τα περιβάλλοντα από τα οποία μπορεί να κληθεί μια εφαρμογή Java δεν υποστηρίζουν ακριβώς τους ίδιους τύπους δεδομένων που υποστηρίζει η Java. Προκύπτει επομένως η ανάγκη να υπάρχει μια διεπαφή ικανή να εκκινήσει μια εφαρμογή Java που να είναι ταυτοχρόνως συμβατή με τα διάφορα περιβάλλοντα κλήσης.

Την ανάγκη αυτή καλύπτει η συνάρτηση main. Η main αποτελεί την κύρια είσοδο σε κάθε εφαρμογή Java. Αυτό σημαίνει πως από οποιοδήποτε περιβάλλον και αν καλέσουμε μια εφαρμογή Java, η εικονική μηχανή θα ψάξει να βρει και να καλέσει την main. Στη συνέχεια, η main μπορεί να καλέσει ανάλογα με τον σχεδιασμό της εφαρμογής οποιαδήποτε συνάρτηση είναι στη διάθεση της εφαρμογής. Επομένως, υπάρχει η ανάγκη να παρέχεται ένας τρόπος με τον οποίο τα διάφορα περιβάλλοντα κλήσης να μεταβιβάζουν παραμέτρους στην main. Ένας τύπος δεδομένων διαθέσιμος σε όλα τα περιβάλλοντα είναι η αλφαριθμητική σειρά. Με τη μορφή αλφαριθμητικής σειράς μπορεί να αναπαρασταθούν δεδομένα οποιουδήποτε τύπου. Για παράδειγμα, μια πραγματική τιμή μπορεί να αναπαρασταθεί ως αλφαριθμητική σειρά. Η τιμή 2.53 ας πούμε, μπορεί να αναπαρασταθεί ως “2.53”. Αυτός είναι ο λόγος που η main λαμβάνει ως παράμετρο έναν πίνακα από Strings. Μέσα από την παράμετρο αυτή, τα περιβάλλοντα κλήσης μπορούν να περάσουν τις παραμέτρους που απαιτούνται. Από τη στιγμή που η main θα έχει τις τιμές των παραμέτρων με τη μορφή αλφαριθμητικών τιμών, έχει τη δυνατότητα να τις μετατρέψει στους κατάλληλους τύπους και να εργαστεί με αυτές χωρίς πρόβλημα.

Ας πάρουμε όμως τα πράγματα από την αρχή. Ας θυμηθούμε τη διεπαφή της main.

```
public static void main(String[] args)
```

Καταρχάς, στην τυπική περίπτωση, η main δηλώνεται ως public. Αυτό είναι λογικό, διαφορετικά θα ήταν αδύνατο να κληθεί από τρίτους. Βέβαια ο μεταγλωττιστής δεν θα διαμαρτυρηθεί αν η main δηλωθεί με πρόσβαση πακέτου. Ενδεχομένως μάλιστα σε κάποιες ειδικές περιπτώσεις, όπου η main καλείται μέσα από Java, μια τέτοια δήλωση να είναι χρήσιμη. Ούτε και για τους υπόλοιπους προσδιοριστές προσπέλασης διαμαρτύρεται ο μεταγλωττιστής. Ωστόσο, σε όλες τις περιπτώσεις που η main θα τρέξει απευθείας από την εικονική μηχανή και όχι μέσα από κάποιον κώδικα Java, θα πρέπει να χρησιμοποιηθεί ο προσδιοριστής προσπέλασης public. Αν παρόλα αυτά επιχειρήσουμε να τρέξουμε μια εφαρμογή με μη δημόσια main, η εικονική μηχανή θα αποκριθεί με κατάλληλο μήνυμα στο οποίο θα αναφέρει ότι δεν βρίσκει main.

Επιπλέον, η main είναι δηλωμένη ως static. Πρόκειται για απαραίτητη δήλωση. Στην περίπτωση που θα δηλωθεί ως μη στατική συνάρτηση, η εικονική μηχανή θα αποκριθεί και πάλι πως δεν βρίσκει main. Η main ως κύρια είσοδος στην εφαρμογή πρέπει απαραίτητα να δηλωθεί ως στατική συνάρτηση, καθώς κλήση των μη στατικών συναρτήσεων μπορεί να πραγματοποιηθεί μόνο μέσα από εφαρμογή Java και άρα έπεται της εκκίνησης της εφαρμογής.

Επίσης, η main πρέπει να είναι τύπου void. Για αυτήν την απαίτηση ίσως να μην υπάρχει αναγκαιότητα. Θα μπορούσε να επιστρέφει int αντί για void όπως η main της C. Στην Java όμως έγινε αυτή η σχεδιαστική επιλογή. Η εικονική μηχανή ψάχνει μια main τύπου void.

Ας δούμε όμως μια πρότυπη main με παραμέτρους.

```
public class MainParameters {

    private static void syntax() {
        System.out.println("MainParameters int String");
        System.exit(1);
    }

    public static void main(String[] prms) {
        if (prms.length != 2) {
            syntax();
        }
        int firstArgument = Integer.parseInt(prms[0]);
        System.out.println("first Argument is " + firstArgument
                + "\nSecond Argument is " + prms[1]);
    }
}
```

Κώδικας 7.10 *Παράδειγμα main με παραμέτρους*

Στον κώδικα 7.10 παρουσιάζουμε την κλάση MainParameters η οποία περιέχει main με παραμέτρους. Βέβαια όλες οι main που είναι κύριες είσοδοι στο πρόγραμμα διαθέτουν τυπική παράμετρο τύπου String[]. Όσες είδαμε όμως μέχρι εδώ δεν χρησιμοποιούν την παράμετρο. Αντίθετα, η MainParameters τη χρησιμοποιεί.

Πιο συγκεκριμένα, η main εδώ χρειάζεται δύο παραμέτρους. Η πρώτη πρέπει να είναι τύπου int και η δεύτερη String. Οι τιμές των δύο παραμέτρων περνάνε στην main ως αλφαριθμητικές σειρές διαμέσου του πίνακα prms. Ας σημειωθεί εδώ πως το αναγνωριστικό prms δεν είναι παρά το αναγνωριστικό μιας παραμέτρου. Μπορεί ο προγραμματιστής που υλοποιεί την main να επιλέξει όποιο αναγνωριστικό κρίνει κατάλληλο. Συνήθως για την παράμετρο της main χρησιμοποιείται το αναγνωριστικό args.

Μέσα στον πίνακα prms λοιπόν αναμένει η main του κώδικα 7.10, δύο παραμέτρους. Η πρώτη που θα τοποθετηθεί στη θέση 0 του πίνακα θα αναπαριστά μια ακέραια τιμή και η δεύτερη που θα τοποθετηθεί στη θέση 1 θα είναι μια αλφαριθμητική σειρά.

Το πρώτο πράγμα που θα πρέπει να ελέγξουμε στην main είναι αν όντως κλήθηκε με σωστές παραμέτρους. Πράγματι, στην πρώτη γραμμή της main ελέγχουμε αν η εφαρμογή κλήθηκε με 2 παραμέτρους. Αν όχι, καλούμε τη συνάρτηση syntax.

Η syntax βγάζει ένα μήνυμα που ενημερώνει τον χρήστη πως η εφαρμογή μας καλείται με δύο παραμέτρους, μία τύπου int και μια τύπου String. Στη συνέχεια καλεί την System.exit με παράμετρο 1. Η System.exit προκαλεί άμεσο τερματισμό της εικονικής μηχανής και άρα και της εφαρμογής που την καλεί. Επομένως αν κληθεί η Syntax βγαίνει ένα μήνυμα στον χρήστη που υποδεικνύει τον ορθό τρόπο κλήσης και η εφαρμογή τερματίζεται καθώς χωρίς τις κατάλληλες παραμέτρους αδυνατεί να προχωρήσει.

Αν δεν κληθεί η syntax, τότε η εφαρμογή επιχειρεί να μετατρέψει την πρώτη παράμετρο, δηλαδή αυτή που βρίσκεται στη θέση μηδέν του prms σε int. Έτσι όπως είναι ο κώδικας 7.10, αν η πρώτη παράμετρος δεν είναι μετατρέψιμη σε int, το πρόγραμμα πέφτει παράγοντας κατάλληλο μήνυμα. Αν η μετατροπή των παραμέτρων γίνει με επιτυχία, τότε η main έχει τις τιμές των παραμέτρων σε τοπικές μεταβλητές και μπορεί να προχωρήσει απρόσκοπτα στην ενδεδειγμένη επεξεργασία. Στον κώδικα 7.10 η πρώτη παράμετρος μετατρέπεται σε int με τη βοήθεια της Integer.parseInt και η τιμή της αποθηκεύεται στην τοπική μεταβλητή firstArgument. Η δεύτερη παράμετρος είναι τύπου String και δεν απαιτείται καμιά μετατροπή. Μπορεί να χρησιμοποιηθεί η prms[1].

Να επανέλθουμε λίγο στην System.exit και στην τιμή 1 που της περάσαμε ως πραγματική παράμετρο. Όπως έχουμε εξηγήσει, οι συναρτήσεις διαθέτουν μια τιμή επιστροφής που σκοπό έχει να ενημερώνει τον κώδικα που κάλεσε τη συνάρτηση με τα αποτελέσματα της κλήσης. Η main όμως είναι εξ ορισμού void. Η παράμετρος της System.exit αποτελεί έναν τρόπο για να ενημερώνουμε αυτόν που καλεί την εφαρμογή για το αποτέλεσμά της. Σε περίπτωση που η εφαρμογή έτρεξε ομαλά, η τιμή της παραμέτρου πρέπει να είναι μηδέν. Οποιαδήποτε άλλη τιμή μπορεί να χρησιμοποιηθεί για να επικοινωνήσει μη ομαλό τερματισμό της εφαρμογής. Από εκεί και μετά, οι πιθανές τιμές είναι θέμα σύμβασης μεταξύ της εφαρμογής και του περιβάλλοντος που την καλεί. Στα Windows για παράδειγμα, η τιμή της παραμέτρου της System.exit ενημερώνει τη μεταβλητή συστήματος errorlevel. Όλα τα scripts των Windows που καλούν κάποια εφαρμογή Java, μπορούν να ελέγχουν την errorlevel και να λαμβάνουν πληροφορίες σχετικά με τον τερματισμό της εφαρμογής.

Ας δούμε όμως δύο παραδείγματα κλήσης της MainParameters. Πιο συγκεκριμένα, θα δούμε πώς καλείται η MainParameters από το NetBeans και από το powershell των Windows.

7.3.4.1 Κλήση από το NetBeans

Στο τμήμα Projects του NetBeans, μεταβείτε στο project στο οποίο έχετε ορίσει την MainParameters και πατήστε δεξί κλικ. Θα ανοίξει ένα πτυσσόμενο μενού. Επιλέξτε Properties. Θα ανοίξει η φόρμα Project Properties.